

Database Architectures

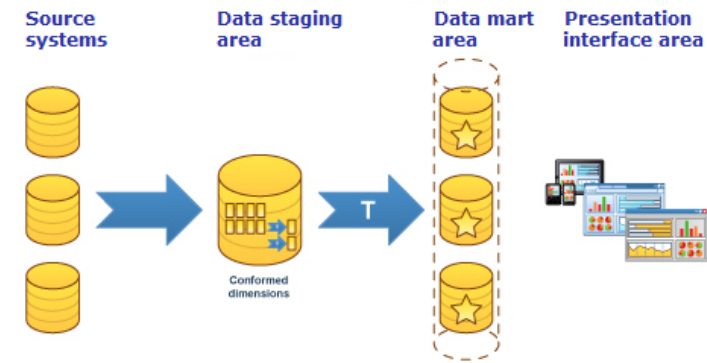
Software System Development
Monsoon 2026

Medallion Architecture

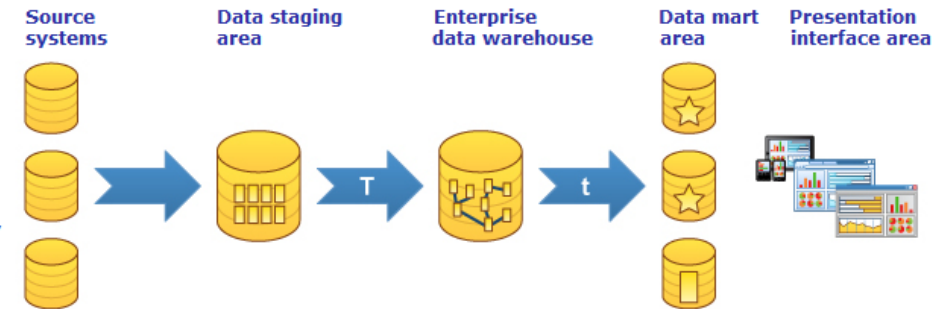


Datawarehousing

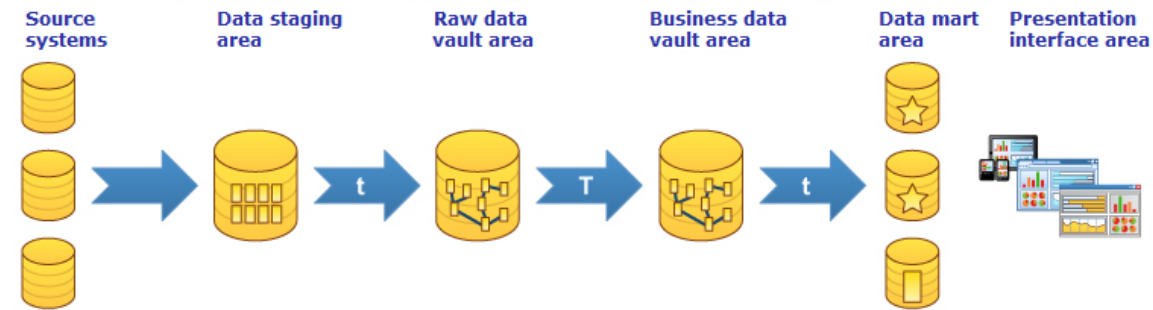
Kimball
Dimensional modeling
Star schema
Dimensional model



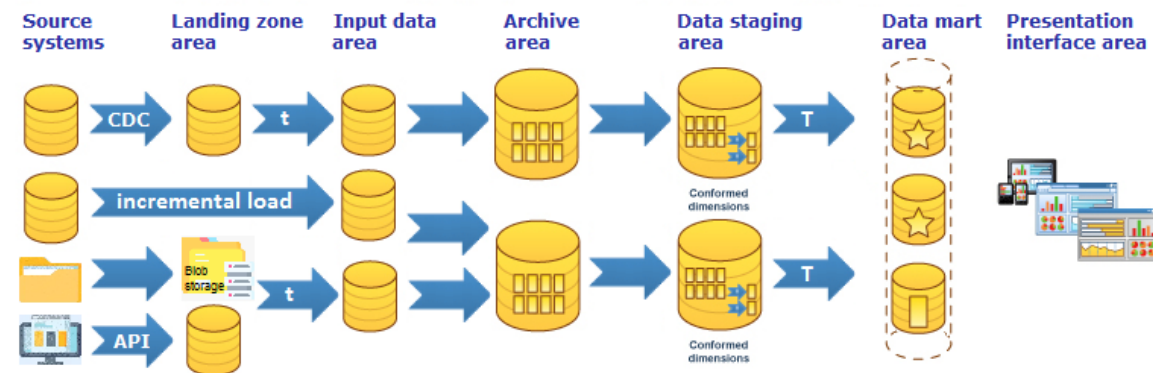
Inmon
Entity Relationship modeling
Corporate Information Factory
Relational model



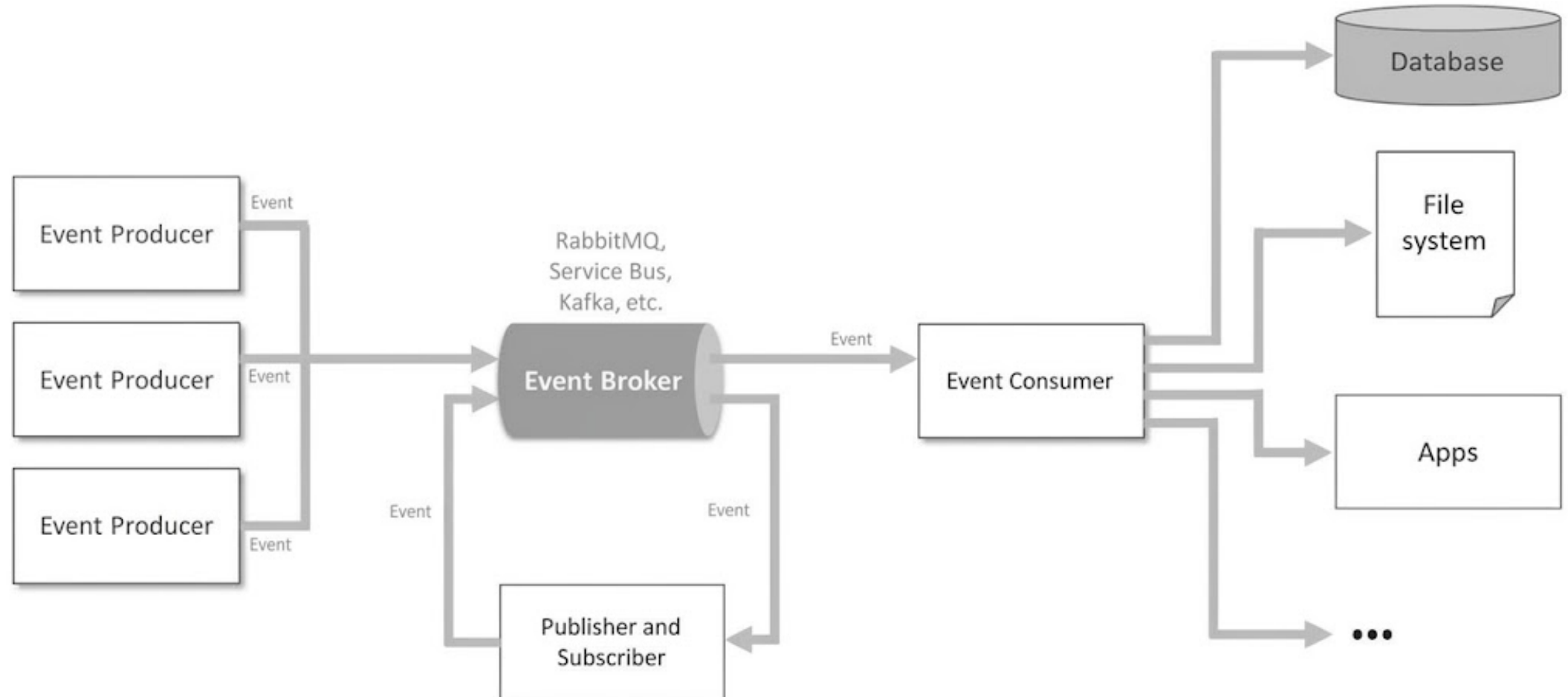
Linstedt
Data vault modeling
Core Business Concept
Data vault model



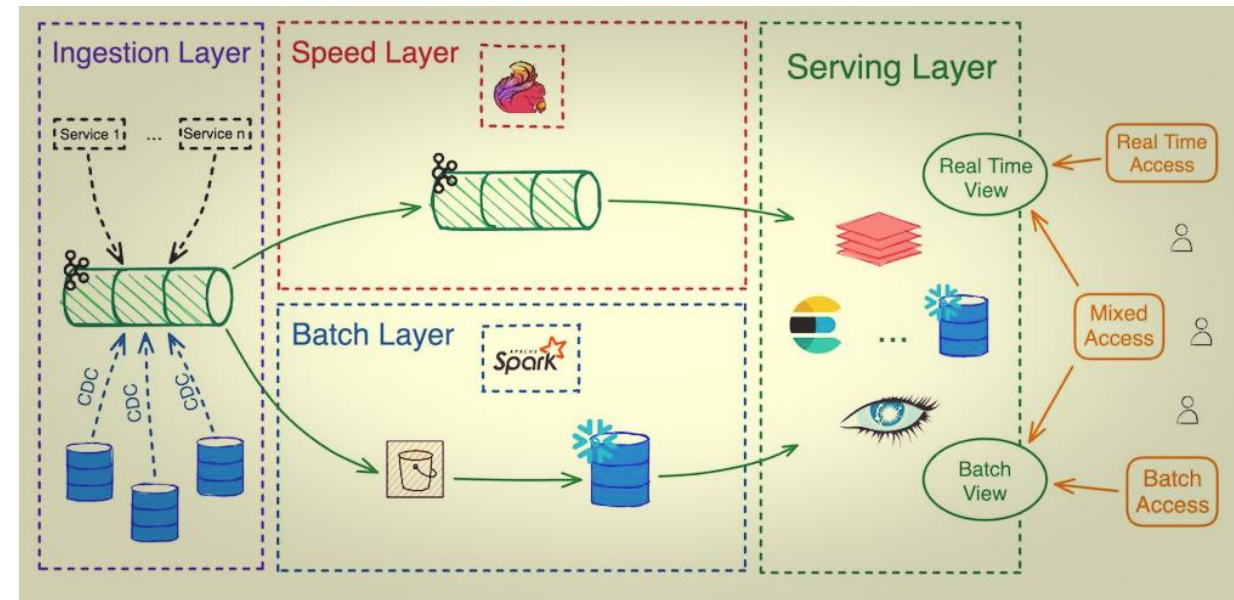
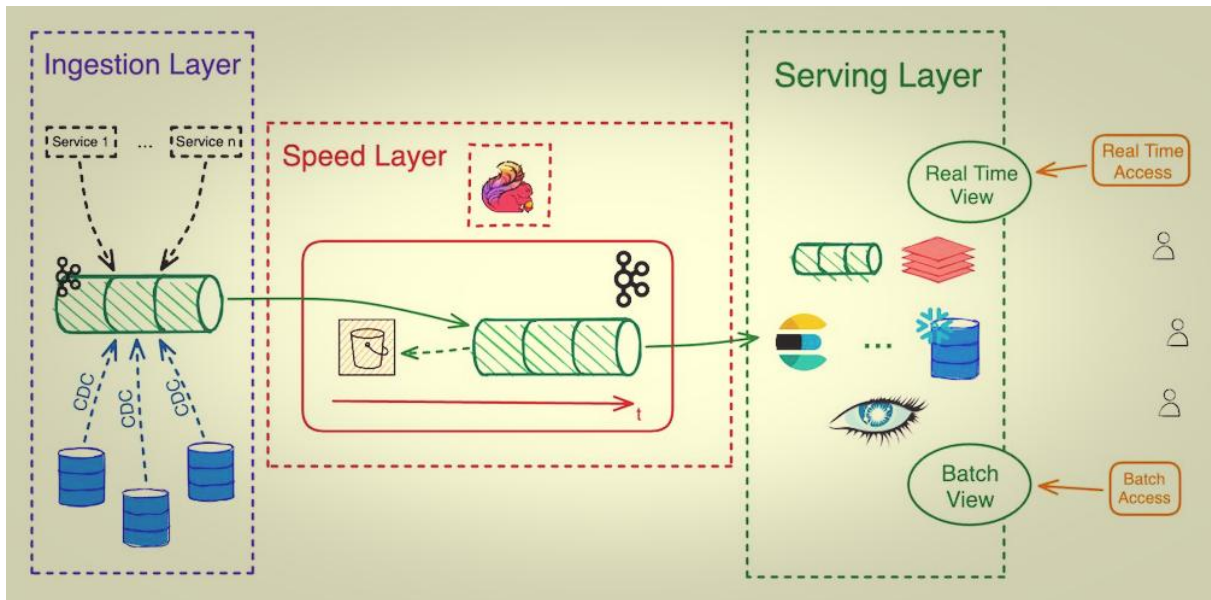
Kimball
Dimensional modeling
Star schema
Dimensional model
Extended by Dalby



Event Driven Architecture



Datalake (Batch – Stream – Ingest)



Writing Semantic Layer

A **semantic layer** is a business-facing abstraction layer that sits between raw data storage (such as data warehouses, data lakes, or relational databases) and business intelligence (BI) tools, analytics applications, or end-users.

Its primary purpose is to transform complex, highly technical database schemas—complete with foreign keys, cryptic table names, and raw data types—into a unified, standardized business vocabulary (e.g., "Revenue," "Active Customer," "Churn Rate").

Single Source of Truth

Performance Optimization

Decoupling

Access Control

How the Code Works

- **Entities & Dimensions:** Define how tables relate to one another (customer_id) and provide categorical or temporal attributes (ordered_at, order_status) for slicing data.
- **Measures:** Define raw aggregations (sum, count, average) directly on database columns.
- **Metrics:** Build business logic on top of measures by applying filters, ratios, or derived calculations (e.g., filtering only for completed orders to calculate total_revenue).
- **Consumption:** Once written, data consumers can query these semantic definitions via SQL, BI connectors, or APIs without needing to write complex JOIN statements or remember table structures.

semantic_models:

- name: fact_orders
description: "Core transactional order data for e-commerce analytics."
model: ref('stg_orders')

entities:

- name: order_id
type: primary
- name: customer_id
type: foreign

dimensions:

- name: ordered_at
type: time
type_params:
time_granularity: day
- name: order_status
type: categorical

measures:

- name: order_amount
agg: sum
- name: unique_customers
agg: count_distinct
expr: customer_id

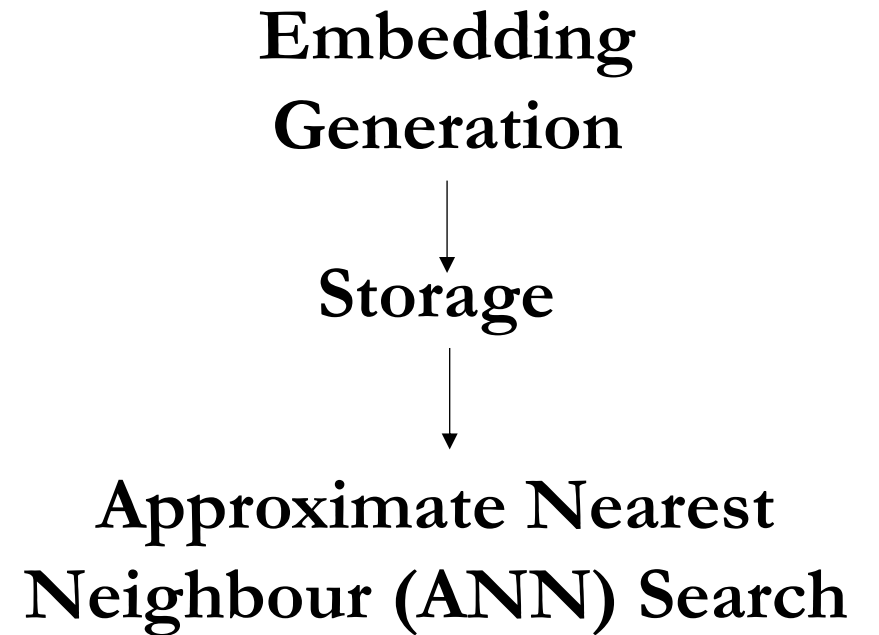
metrics:

- name: total_revenue
description: "Total gross revenue generated from completed orders."
type: simple
label: "Total Revenue"
type_params:
measure: order_amount
filter: "{{
Dimension('fact_orders__order_status') }}" =
'completed'"
- name: active_customer_count
description: "Count of unique active purchasing customers."
type: simple
label: "Active Customers"
type_params:
measure: unique_customers

Vector Databases

A vector database is a specialized data management system designed to store, index, and query high-dimensional vectors.

Unlike traditional databases that query data using exact matches, keywords, or relational filters, a vector database queries data based on semantic similarity.



Distance Metrics: Cosine Similarity, Euclidean Distance, Dot Product

```
SELECT id, content
FROM documents
WHERE category = 'engineering'
ORDER BY embedding <=> '[0.12, -0.43, 0.58, ...]'::vector
LIMIT 5;
```

```
results = index.query(
    vector=[0.12, -0.43, 0.58, ...],
    top_k=5,
    include_metadata=True,
    filter={"category": {"$eq": "engineering"}}
)
```